

後輩に ROS 2 を教える羽目になった学生が、採点をやめた話

ros2-cpp-drill — テストが採点するので、採点する人が要らない

nyanziba / github.com/nyanziba/ros2-cpp-drill

去年の自分が、ここで詰まっていた

ROS 2 を始めたとき、公式チュートリアルの
このコードが読めませんでした。

```
publisher_ = create_publisher<String>("topic", 10);  
timer_ = create_wall_timer(  
    500ms, std::bind(&Node::cb, this));
```

止まった原因は ROS ではありません。

テンプレート・`shared_ptr`・`std::bind` でした。

「ROS を学んでいるのか、
C++ の文法と戦っているのか
分からない」

当時これを言語化できず、 とりあえず写経して先に進みました。

今年、教える側に回って詰まり直した

採点が回らない

課題を出しても、**自分が見ないと合否が出ない**。
新人は「これで合ってるのか」が分からないまま
進んでしまう。

同じ質問が何度も来る

しかも自分は、去年その質問をしていた側です。
答えは分かるのに、**人数分の時間がない**。

採点する人が要らない 課題にすればいい

Rust の rustlings がまさにそれをやっていたので、同じ
形を ROS 2 に持ってきました。

作ったもの: ros2-cpp-drill

Rust の **rustlings 方式** を ROS 2 に持ち込んだ練習帳。

- 穴埋めのソースを埋めると、**テストが合否を判定する**
- 読み物（docs/ 49章）と課題（exercises/ 35問）が**章番号まで1対1**
- 課題は全部 gtest / pytest。 **GUI 無しで最後まで通せる**

```
./drill watch    # 保存するたびに再テストされる
```

トラック	課題	ROS 2	ねらい
C++入門編	cppb01 ~ cppb10	使わない	const ・ static で手が止まる人
C++編	cpp01 ~ cpp12	使わない	rclcpp を読む準備
ROS 2編	01 ~ 15	使う	pub/sub から composition まで

なぜ C++ を先に片付けるのか

rclcpp のコードは、C++ の道具の上に建っています。

```
publisher_ = create_publisher<String>("topic", 10);
timer_ = create_wall_timer(
    500ms, std::bind(&Node::cb, this));
```

この 3 行を読むのに要るもの:

- テンプレート引数
- `shared_ptr` (`publisher_` の型)
- `std::bind` とメンバ関数ポインタ
- `chrono` のリテラル

C++入門編と C++編は ROS 2 を使いません。

`ament_cmake` と `gtest` だけなので、
1 課題 2~3 秒でビルドできます。

ROS 2 の課題はビルドに 10~40 秒かかります。文法の練習をそこでやると、待ち時間で心が折れます。

**ここが曖昧なまま ROS 2 に入ると、
何と戦っているのか分からなくなる。**

課題一覧と進捗

```
ros2-drill - 課題一覧と進捗

$ ./drill list
ROS 2 練習帳

【C++入門編】 docs/cpp-basics/ の講習に対応。ROS 2 は使いません
▶cppb01_declarations 宣言を読む [入門 1章] ←今ここ
  cppb02_scope       スコープと寿命 [入門 2章]
  cppb03_reference    参照で受け取る [入門 3章]
  ... (中略) ...

【初級】 講習資料 11~14章
✓ 01_publisher       トピックに publish する [11章]
  02_subscriber       トピックを購読する [11章]
  03_custom_interface カスタムインターフェースを定義して使う [13章]
  04_service_server   サービスサーバを作る [14章]
  05_service_client   サービスクライアントを作る [14章]

【中級】 講習資料 15~17章
  06_parameters       クラスの中でパラメータを使う [15章]
  07_param_events      パラメータの変更を監視する [15章]
  08_params_yaml       パラメータを YAML で管理する [15章]
  09_launch            launch を Python / XML / YAML で書く [09章 / 15章]
  10_action_server     アクションサーバを作る [16章]
  11_node_test         テストを書く [17章]

【上級】 資料が「別記事」として先送りしている領域
  12_qos               QoS プロファイルを設計する [05章の発展]
  13_executors          Executor とコールバックグループ [14章の発展]
  14_zero_copy          ゼロコピー (unique_ptr とプロセス内通信) [04章の発展]
  15_composition        コンポーネント化する (RCLCPP_COMPONENTS) [04章の発展]

進捗: 1 / 35
[ ] 内が対応する講習資料の章です。./drill read ID でその章を開けます。
```

- ▶ **がいま解く課題**
- ✓ **が突破済み**
- [11章] **が対応する読み物の章**
→ `./drill read 01` でその章が開く

全 35 問。C++入門編 → C++編 → ROS 2編 の順に並んでいる。

読み物は GitHub Pages に出してある



nyanziba.github.io/ros2-cpp-drill — 日本語の全文検索つき、インストール不要。

課題からその章に飛べる



`./drill read 01` で開くのがこの章。課題 `01_publisher` と1対1で対応している。

ここから実演

課題 01 「トピックに publish する」を実際に解く

課題 01: 与えられるのはこれだけ

exercises/01_publisher/src/minimal_publisher.cpp

```
MinimalPublisher::MinimalPublisher()  
: Node("minimal_publisher"), count_(0)  
{  
    // TODO: "topic" に std_msgs::msg::String を流す Publisher を作り、publisher_ に入れること。  
    //      QoS の depth は 10。使う API は create_publisher。  
  
    // TODO: 500ms 周期のタイマを作り、timer_ に入れること。  
    //      呼び出すのは下の timer_callback。使う API は create_wall_timer で、  
    //      メンバ関数を渡すには std::bind が要る。  
}
```

クラス宣言（.hpp）と CMakeLists は与えてある。 考えるのは中身だけ。

仕様は公式チュートリアルの `MinimalPublisher` と同一。

まず、埋めずに走らせてみる

```
未解答のまま走らせる — テストが「次にやること」を言う

$ ./drill run 01
——初級 01_publisher: トピックに publish する ——

ビルド中...
[ RUN      ] DrillTest.topicトピックにpublishしている
~/ros2-cpp-drill/exercises/01_publisher/test/test_exercise.cpp:42: Failure
Value of: drill::spin_until({talker, probe.node}, [&probe]() {return probe.received.size() >= 2;}, 8s)
  Actual: false
 Expected: true
"topic" に 8 秒待っても 2 件届きませんでした (受信 0 件)。
- create_publisher<std_msgs::msg::String>("topic", 10) を publisher_ に入れましたか？
- create_wall_timer(500ms, ...) を timer_ に入れましたか？
- タイマのコールバックで publisher_->publish(message) を呼んでいますか？

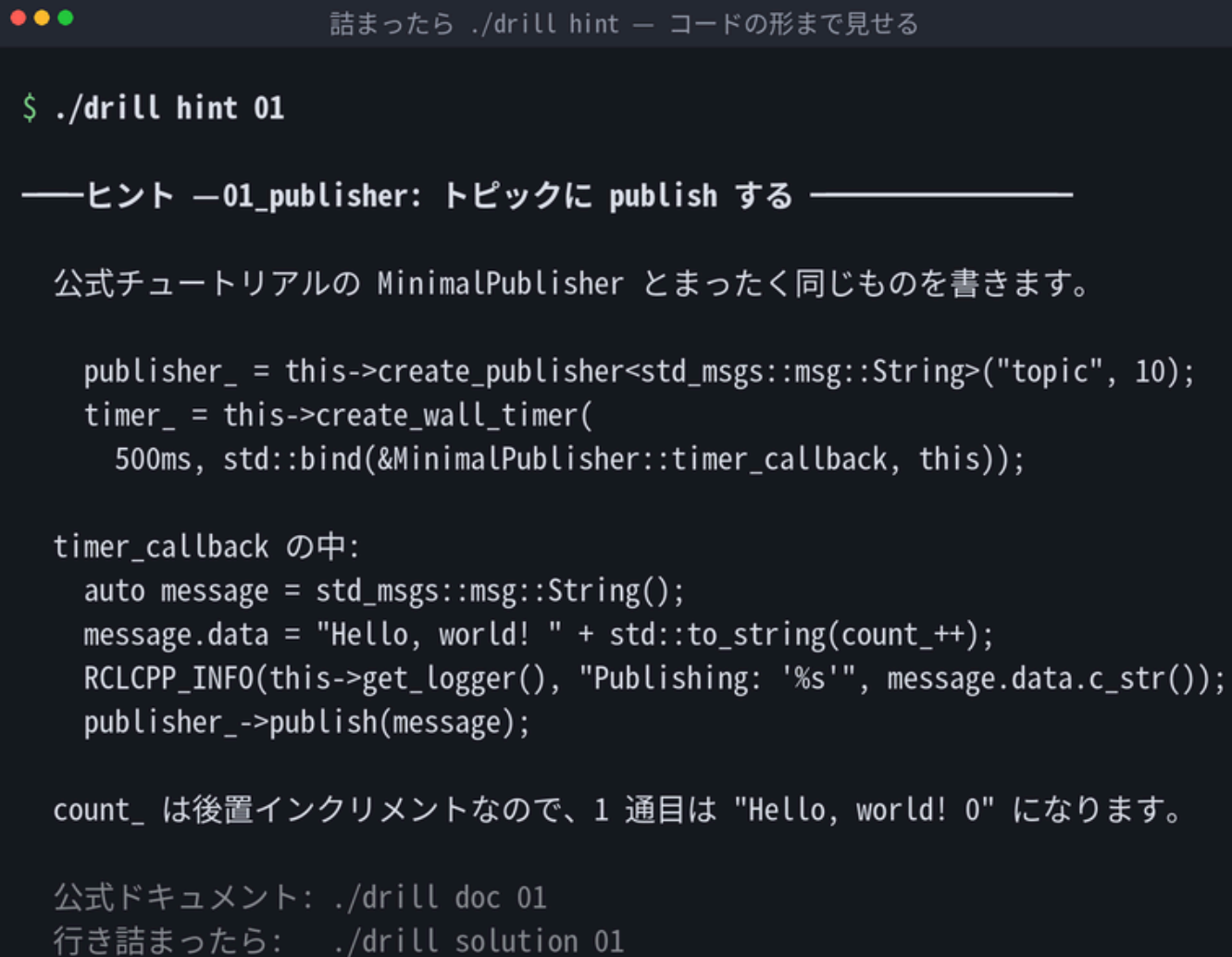
[ FAILED   ] DrillTest.topicトピックにpublishしている (8052 ms)
.....(中略).....
[ PASSED   ] 0 tests.
.....(中略).....

4 FAILED TESTS

× 課題 01_publisher は未完了です
  課題文: ~/ros2-cpp-drill/exercises/01_publisher/README.md
  編集する: ~/ros2-cpp-drill/exercises/01_publisher/src/minimal_publisher.cpp
  ヒント:  ./drill hint 01
```

落ちたテストが「次にやること」を日本語で返す。人に聞かなくていい。

詰まったら `./drill hint`



```
$ ./drill hint 01

——ヒント ——01_publisher: トピックに publish する ——

公式チュートリアルの MinimalPublisher とまったく同じものを書きます。

publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
timer_ = this->create_wall_timer(
    500ms, std::bind(&MinimalPublisher::timer_callback, this));

timer_callback の中:
    auto message = std_msgs::msg::String();
    message.data = "Hello, world! " + std::to_string(count_++);
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);

count_ は後置インクリメントなので、1 通目は "Hello, world! 0" になります。

公式ドキュメント: ./drill doc 01
行き詰まったら:   ./drill solution 01
```

それでも駄目なら `./drill solution 01`。答えを見ることは禁止していない。

手が止まったときの逃げ道は4段階

```
./drill doc — 対応する章と公式ドキュメントの URL

$ ./drill doc 01

——参考資料 —01_publisher: トピックに publish する——

対応する講習資料
11_C++でpub_subを書く (publisher を書く節)
~/ros2-cpp-drill/docs/ros2/11_C++でpub_subを書く.md

公式ドキュメント
Writing a simple publisher and subscriber (C++)
https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html
```

hint (コードの形) → doc (公式 URL) → read (対応する章) → solution (解答例)。

埋める

```
MinimalPublisher::MinimalPublisher()
: Node("minimal_publisher"), count_(0)
{
    publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
    timer_ = this->create_wall_timer(
        500ms, std::bind(&MinimalPublisher::timer_callback, this));
}

void MinimalPublisher::timer_callback()
{
    auto message = std_msgs::msg::String();
    message.data = "Hello, world! " + std::to_string(count_++); // ← ここ
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);
}
```

わざと間違えてみる: `count_++` → `++count_`

```
間違えたとき — 何かがどう違うかを日本語で返す

$ ./drill run 01 # count_++ を ++count_ と書いてしまった
[ OK ] DrillTest.topicトピックにpublishしている (1059 ms)
.....(中略).....
~/ros2-cpp-drill/exercises/01_publisher/test/test_exercise.cpp:62: Failure
Expected equality of these values:
  probe.received[1]
    Which is: "Hello, world! 2"
  "Hello, world! 1"
2 通目が "Hello, world! 1" になっていません。count_ を後置インクリメント (count_++) していますか？ 実際の値: "Hello, world! 2"
.....(中略).....
[ FAILED ] DrillTest.本文がHello_worldと連番になっている (1529 ms)
[ PASSED ] 2 tests.
```

4問中 **2問は通る**。落ちた2問が「**後置インクリメントしていますか？**」と聞いてくる。

直すと通る

```
埋まると通る — 次に進む合図は「I AM NOT DONE を消す」

$ ./drill run 01
——初級 01_publisher: トピックに publish する ——

ビルド中...

....(中略)....
[ RUN      ] DrillTest.topicトピックにpublishしている
[INFO] [1785597615.616233108] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1785597616.116240063] [minimal_publisher]: Publishing: 'Hello, world! 1'
....(中略)....
[ OK      ] DrillTest.本文がHello_worldと連番になっている (1517 ms)
[ RUN      ] DrillTest.おおよそ500ミリ秒周期でpublishしている
[INFO] [1785597618.158424806] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1785597618.658456611] [minimal_publisher]: Publishing: 'Hello, world! 1'
[INFO] [1785597619.158428630] [minimal_publisher]: Publishing: 'Hello, world! 2'
[ OK      ] DrillTest.おおよそ500ミリ秒周期でpublishしている (1520 ms)
[ RUN      ] DrillTest.公式と同じPublishingログを出している
[ OK      ] DrillTest.公式と同じPublishingログを出している (518 ms)
[-----] 4 tests from DrillTest (4604 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (4604 ms total)
[ PASSED ] 4 tests.

✓ すべてのテストに合格しました！

次に進むには、下の行を削除してください:
// I AM NOT DONE
(~/ros2-cpp-drill/exercises/01_publisher/src/minimal_publisher.cpp)
```

通っただけでは次に進まない。 **// I AM NOT DONE** を自分で消すのが完了の合図。

実際の運用: `./drill watch` を開いたまま書く

```
./drill watch — 保存するたびに再テストし、通ったら次の課題へ

$ ./drill watch 01
watch モード (Ctrl-C で終了)
監視中: ~/ros2-cpp-drill/exercises/01_publisher/src/minimal_publisher.cpp
.....(中略).....
——初級 01_publisher: トピックに publish する ——

ビルド中...

.....(中略).....
× 課題 01_publisher は未完了です
.....(中略).....
保存すると自動で再テストします...
.....(中略).....
[ PASSED ] 4 tests.

✓ すべてのテストに合格しました！
  次の課題: ./drill run cppb01 (宣言を読む)

次の課題に移りました: cppb01_declarations
```

保存 → 自動で再ビルド → 再テスト → 通ったら**次の課題へ勝手に進む**。

テストが通ったコードは、そのまま実機で動く

```

$ ros2 run drill_01_publisher talker
[INFO] [1785597663.348667125] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1785597663.848637972] [minimal_publisher]: Publishing: 'Hello, world! 1'
[INFO] [1785597664.348659382] [minimal_publisher]: Publishing: 'Hello, world! 2'
[INFO] [1785597664.848666892] [minimal_publisher]: Publishing: 'Hello, world! 3'
[INFO] [1785597665.348668951] [minimal_publisher]: Publishing: 'Hello, world! 4'
[INFO] [1785597665.848634622] [minimal_publisher]: Publishing: 'Hello, world! 5'

$ ros2 topic echo /topic
data: Hello, world! 8
---
data: Hello, world! 9
---
data: Hello, world! 10
---
data: Hello, world! 11
---

$ ros2 topic hz /topic
average rate: 2.000
      min: 0.500s max: 0.500s std dev: 0.00015s window: 4
average rate: 2.000
      min: 0.500s max: 0.500s std dev: 0.00012s window: 7

```

`ros2 topic hz` が **2.000 Hz**。500ms 周期のタイマがそのとおり動いている。

出来上がるのは、ドリル用の何かではない

```
出来上がるのは普通の ROS 2 ノード

$ ros2 node info /minimal_publisher
/minimal_publisher
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /topic: std_msgs/msg/String
Service Servers:
  /minimal_publisher/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /minimal_publisher/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /minimal_publisher/get_parameters: rcl_interfaces/srv/GetParameters
```

`ros2 node info` で見ても普通のノード。`ros2 run` も `rqt` も普段どおり使える。

採点している中身

テストは probe ノードを立てて外から観測する

```
subscription = node->create_subscription<
  std_msgs::msg::String>(
    "topic", 10,
    [this](std_msgs::msg::String::ConstSharedPtr msg) {
      received.push_back(msg->data);
      stamps.push_back(
        std::chrono::steady_clock::now());
    });
```

実装の中身は覗かない。

トピックに何が流れたかだけを見る。

だから測れること

- 本文と連番（`count_++` の後置）
- 周期が約 500ms か（`stamps` の差分）
- 公式と同じログが出ているか

CI が毎回、両方向を確認

- 未解答の課題が**ちゃんと落ちる**
- 解答を当てた課題が**ちゃんと通る**

「テストがザルで空でも通る」を防ぐ。

rclcpp のテストで踏んだところ

待ち方

`sleep` で待つとテストが不安定になります。

`spin_until` で「条件が満たされるまで回す」形にそろえてあります。

```
drill::spin_until(  
    {talker, probe.node},  
    [&]{ return probe.received.size() >= 2; },  
    8s);
```

名前の衝突

同じ名前のノードを並行して立てると壊れます。

`drill::unique_name()` で毎回ずらしています。

ログの検査

`RCLCPP_INFO` の出力は `drill::LogCapture` で拾って文字列として突き合わせています。

この3つを `tools/drill_harness.hpp` に閉じ込めて、課題側のテストからは1行で使えるようにしました。

課題を1問足すときにやること

`exercises.json` に1エントリ足して、テンプレートと解答を置くだけです。

```
{
  "id": "cppb01_declarations",
  "package": "drill_cppb01_declarations",
  "level": "C++入門編",
  "title": "宣言を読む",
  "kind": "gtest",
  "sources": ["src/reader.cpp"],
  "hints": ["宣言は右から左に読みます。const int * は「const int へのポインタ」。"],
  "lecture": [{ "title": "1. 宣言を読む", "path": "docs/cpp-basics/01_宣言を読む.md" }],
  "chapter": "入門 1章"
}
```

`hints` が `./drill hint`、`lecture` が `./drill read`、`chapter` が一覧の [入門 1章] になります。

ランナーは Python 3 の標準ライブラリだけで書いてあります（追加インストール無し）。

読み物側で決めていること

予想させてから答えを見せる

コード例には「**予想: 出力は何か**」が付いていて、実行結果は `<details>` で畳んであります。外したところがその人の穴なので、飛ばさせないためです。

ブラウザで動かせる

C++ の章のコード例には Compiler Explorer のリンクが付いています。 `g++` が無くても読み進められます。

載せる出力は実測だけ

コンパイルエラーも実行結果も、すべて実際に走らせた出力です。

予想と実測が食い違った箇所は、実測に合わせてあります。

環境は Ubuntu 24.04 / g++ 13.3.0 / ROS 2 Jazzy。この発表の端末画像も全部そうやって作りました。

後輩に渡すときにやること

環境

```
docker compose build
docker compose run --rm drill ./drill watch
```

Ubuntu 24.04 / ROS 2 Jazzy が入っていれば
./drill watch だけ。手元の Mac の人にも
そのまま渡せます (arm64 でも動く)。

CI が確かめていること

- イメージが組めるか
- **未解答の課題が落ちるか**
- 解答を当てた課題が通るか
- ROS 2 の課題もコンテナ内で動くか
- 生成物が root 所有にならないか

手元に Docker が無い環境で書いたので、Docker の検証は CI に寄せています。

扱う範囲と、扱わない範囲

扱う

C++ と ROS 2 の基礎から、
rclcpp のコードを読み書きできるところまで。

ROS 2編は第3部まで。基礎・パッケージ開発・
TF2・URDF・`ros2_control`・センサ統合。

「読める」までを引き受けて、その先は現場のコードで、という切り方です。

扱わない

自律移動スタックそのものの設計
(プランナやコントローラの実装)。

題材が個別のロボットに強く依存するので、この教材の外に置いています。

最後に

自分が答えなくていい状態を作る。これが作りたかった一点です。
落ちたテストが次の一手まで言うので、後輩は自分を待たなくて済みます。

C++ を先に片付ける。去年の自分がそこで止まったからです。
rclcpp は `shared_ptr` とラムダの上に建っています。

公式と同じ字面を書かせる。オリジナル問題を作ると、
解けても公式ドキュメントが読めるようにはなりません。

github.com/nyanziba/ros2-cpp-drill (MIT)